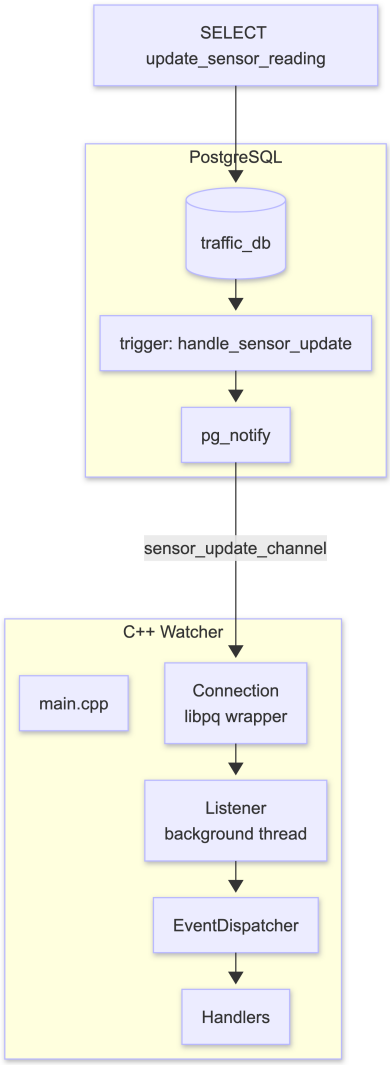
README

SQL Watcher - Architecture Documentation

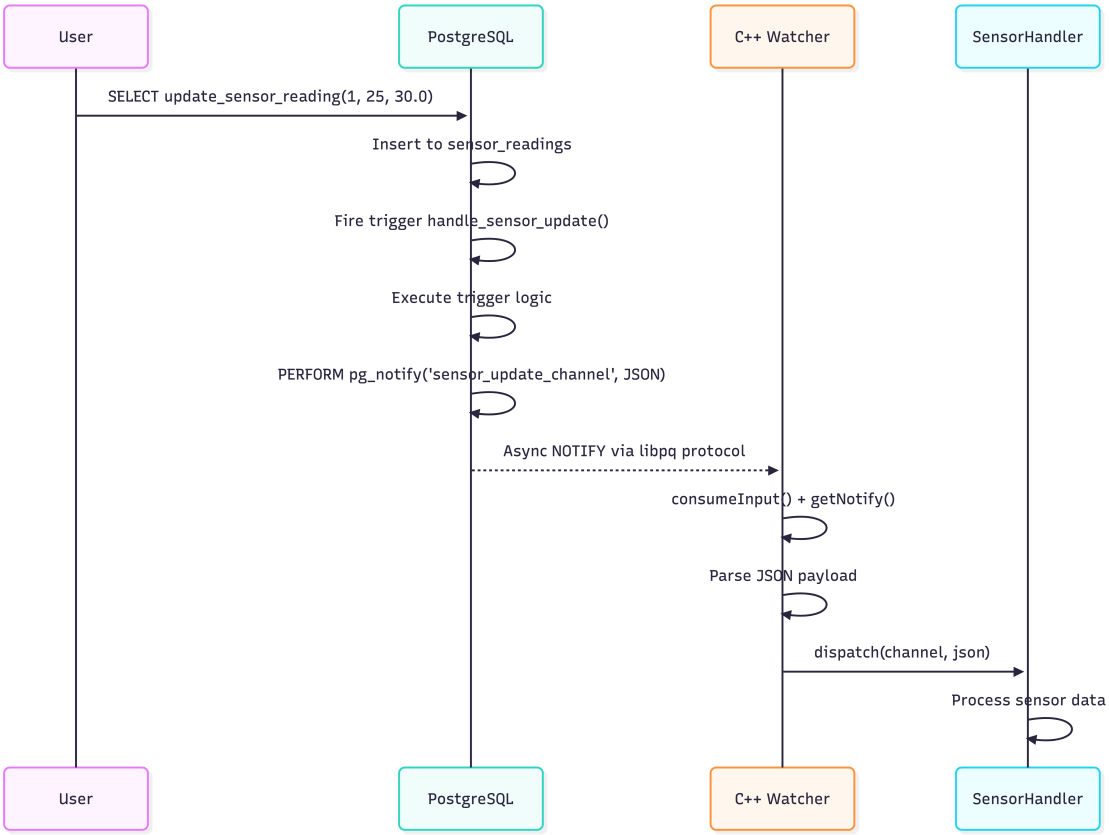
Overview

This C++ application connects to PostgreSQL and listens for NOTIFY events, enabling an “active database” pattern where the C++ app can react to database changes in real-time.

Architecture Diagram



Data Flow



Layer-by-Layer Explanation

Layer 0: SQL Trigger

```
traffic_db=# \sf handle_sensor_update
CREATE OR REPLACE FUNCTION public.handle_sensor_update()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE v_approach_id INT;
BEGIN
    SELECT approach_id INTO v_approach_id
    FROM sensors WHERE sensor_id = NEW.sensor_id;

    UPDATE sensors
    SET vehicle_count = NEW.vehicle_count,
        last_updated = NEW.reading_time
    WHERE sensor_id = NEW.sensor_id;

    UPDATE approaches
    SET queue_count = NEW.vehicle_count,
        last_updated = NEW.reading_time
    WHERE approach_id = v_approach_id;

    PERFORM log_traffic_history(v_approach_id, NEW.vehicle_count, NEW.reading_time);
    PERFORM recompute_phase_for_approach(v_approach_id);

    PERFORM pg_notify(
        'sensor_update_channel',
        json_build_object(
            'type', 'SENSOR_UPDATE',
            'sensor_id', NEW.sensor_id,
            'vehicle_count', NEW.vehicle_count,
            'avg_speed', NEW.avg_speed,
            'reading_time', NEW.reading_time,
            'approach_id', v_approach_id
        )::text
    );

    RETURN NEW;
END;
$function$
traffic_db=#
```

Layer 1: Connection (libpq wrapper)

File: `include/watcher/connection.hpp` , `src/connection.cpp`

Purpose: Connect to PostgreSQL and execute SQL commands.

```
class Connection {
    std::unique_ptr<PGconn, PGconnDeleter> conn_;  // libpq handle

    struct PGconnDeleter {
        void operator()(PGconn* p) { PQfinish(p); }  // Auto-cleanup
    };
};
```

Key methods:

- `execute(sql)` - Run any SQL command
- `listen(channel)` - Subscribe to NOTIFY channel: `LISTEN channel`
- `consumeInput()` - Poll PostgreSQL for new data
- `getNotify()` - Retrieve pending notifications

RAII Pattern: The `unique_ptr` with custom deleter ensures `PQfinish()` is called automatically when the Connection object is destroyed, preventing memory leaks.

Layer 2: Listener (background thread)

File: `include/watcher/listener.hpp` , `src/listener.cpp`

Purpose: Continuously poll for NOTIFY events in a separate thread.

```
class Listener {
    std::thread thread_;           // Background thread
    std::atomic<bool> running_;    // Graceful shutdown flag

    void loop() {
        while (running_) {
            conn_.consumeInput();              // Poll PG
            while (auto* n = conn_.getNotify()) { // Get all pending
                processNotification(n);         // Handle each
            }
            std::this_thread::sleep_for(1ms); // Brief pause
        }
    }

    void processNotification(PGnotify* notify) {
        std::string channel = notify->relname;   // e.g., "sensor_update_channel"
        std::string payload = notify->extra;      // JSON payload
```

```
        dispatcher_.dispatch(channel, payload);
        PQfreemem(notify);
    }
};
```

Thread Safety: The `atomic<bool> running_` flag allows safe signaling between threads without locks.

Layer 3: Dispatcher (handler pattern)

File: `include/watcher/dispatcher.hpp` , `src/dispatcher.cpp`

Purpose: Route events to appropriate handlers based on channel name.

```
class EventDispatcher {
    std::map<std::string, HandlerPtr> handlers_;

    void dispatch(const std::string& channel, const std::string& payload) {
        auto it = handlers_.find(channel);
        if (it != handlers_.end()) {
            json data = json::parse(payload);  // Parse JSON
            it->second->handle(channel, data);
        }
    }
};

class EventHandler {
    virtual void handle(const std::string& channel, const json& data) = 0;
};
```

Extensibility: Add new handlers by simply calling `registerHandler("new_channel", make_unique<NewHandler>())` .

Handlers

File: `include/watcher/handlers/sensor_handler.hpp`

Purpose: Process specific event types.

```
class SensorHandler : public EventHandler {
    void handle(const std::string& channel, const json& data) override {
        // data contains:
        // {"type":"SENSOR_UPDATE","sensor_id":1,"vehicle_count":25,
        //  "avg_speed":30.00,"reading_time":"2026-04-08T18:44:45",...}

        int sensorId = data["sensor_id"];
        int count = data["vehicle_count"];
```